EXPERIMENT NO : 22

AIM : Loading of data and preprocessing using numpy.

Preprocessing the Data

Data preprocessing steps

Follow these steps to preprocess the data in Python:

Step 1: Importing the useful packages: If we are using Python then this would be the first step for converting the data into a certain format, i.e., preprocessing. It can be done as follows:

```
import numpy as np
from sklearn import preprocessing
```

Step 2: Defining sample data: After importing the packages, we need to define some sample data so that we can apply preprocessing techniques on that data. We will now define the following sample data:

```
Input_data = np.array([2.1, -1.9, 5.5],
                      [-1.5, 2.4, 3.5],
                      [0.5, -7.9, 5.6],
                      [5.9, 2.3, -5.8]])
```

Step3: Applying preprocessing technique: In this step, we need to apply any of the preprocessing techniques.

Techniques for Data Preprocessing

Binarization

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array([[2.1,-1.9,5.5],
                       [-1.5,2.4,3.5],
                       [0.5,-7.9,5.6],
                       [5.9,2.3,-5.8]])
data_binarized=preprocessing.Binarizer(threshold=0.5).transform(Input_data)
print("\n Binarized data:\n",data_binarized)
```

Mean Removal

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array([[2.1,-1.9,5.5],
```

```
                [-1.5,2.4,3.5],
                [0.5,-7.9,5.6],
                [5.9,2.3,-5.8]])
print("Mean=",Input_data.mean(axis=0))
print("Std deviation=",Input_data.std(axis=0))
```

The below code will remove the Mean and Standard deviation of the input data :

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array([[2.1,-1.9,5.5],
                [-1.5,2.4,3.5],
                [0.5,-7.9,5.6],
                [5.9,2.3,-5.8]])
data_scaled=preprocessing.scale(Input_data)
print("Mean=",data_scaled.mean(axis=0))
print("Std deviation=",data_scaled.std(axis=0))
```

Scaling

Min max scaling

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array([[2.1,-1.9,5.5],
                [-1.5,2.4,3.5],
                [0.5,-7.9,5.6],
                [5.9,2.3,-5.8]])
data_scaler_Minmax=preprocessing.MinMaxScaler(feature_range=(0,1))
data_scaled_Minmax=data_scaler_Minmax.fit_transform(Input_data)
print("\nminmax scaled data:\n",data_scaled_Minmax)
```

Normalization

L1 Normalization

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array([[2.1,-1.9,5.5],
                [-1.5,2.4,3.5],
                [0.5,-7.9,5.6],
                [5.9,2.3,-5.8]])
data_normalized_l1=preprocessing.normalize(Input_data,norm='l1')
print("\nL1 normalized data:\n",data_normalized_l1)
```

L2 Normalization

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array([[2.1,-1.9,5.5],
                       [-1.5,2.4,3.5],
                       [0.5,-7.9,5.6],
                       [5.9,2.3,-5.8]])
data_normalized_l2=preprocessing.normalize(Input_data,norm='l2')
print("\nL2 normalized data:\n",data_normalized_l2)
```

RESULT : Loading of data and preprocessing using numpy executed successfully.

OUTPUT :

Binarized data:
[[1. 0. 1.]
[0. 1. 1.]
[0. 0. 1.]
[1. 1. 0.]]

Mean= [1.75 -1.275 2.2]
Std deviation= [2.71431391 4.20022321 4.69414529]

Mean= [1.11022302e-16 0.00000000e+00 0.00000000e+00]
Std deviation= [1. 1. 1.]

```
minmax scaled data:
 [[0.48648649 0.58252427 0.99122807]
 [0.         1.         0.81578947]
 [0.27027027 0.         1.        ]
 [1.         0.99029126 0.        ]]
```

```
L1 normalized data:
 [[ 0.22105263 -0.2         0.57894737]
 [-0.2027027   0.32432432  0.47297297]
 [ 0.03571429 -0.56428571  0.4       ]
 [ 0.42142857  0.16428571 -0.41428571]]
```

```
L2 normalized data:
 [[ 0.33946114 -0.30713151  0.88906489]
 [-0.33325106  0.53320169  0.7775858 ]
 [ 0.05156558 -0.81473612  0.57753446]
 [ 0.68706914  0.26784051 -0.6754239 ]]
```

EXPERIMENT NO : 23

AIM : Installing and familiarising weka in ubuntu.

LINUX #

- Click here to download a zip archive for Linux that includes Azul's 64-bit OpenJDK Java VM 17 (weka-3-9-6-azul-zulu-linux.zip; 146.7 MB)

First unzip the zip file. This will create a new directory called weka-3-9-6. To run Weka, change into that directory and type

```
./weka.sh
```

OTHER PLATFORMS

- Click here to download a zip archive containing Weka (weka-3-9-6.zip; 59.4 MB)

First unzip the zip file. This will create a new directory called weka-3-9-6. To run Weka, change into that directory and type

```
java -jar weka.jar
```

Note that Java needs to be installed on your system for this to work. Also note, that using `-jar` will override your current CLASSPATH variable and only use the `weka.jar`.

RESULT : Installing and familiarising weka in ubuntu executed successfully.

EXPERIMENT NO : 24

AIM : Loading of data using weka.

In this chapter, we start with the first tab that you use to preprocess the data. This is common to all algorithms that you would apply to your data for building the model and is a common step for all subsequent operations in WEKA.

For a machine learning algorithm to give acceptable accuracy, it is important that you must cleanse your data first. This is because the raw data collected from the field may contain null values, irrelevant columns and so on.

In this chapter, you will learn how to preprocess the raw data and create a clean, meaningful dataset for further use.

First, you will learn to load the data file into the WEKA explorer. The data can be loaded from the following sources –

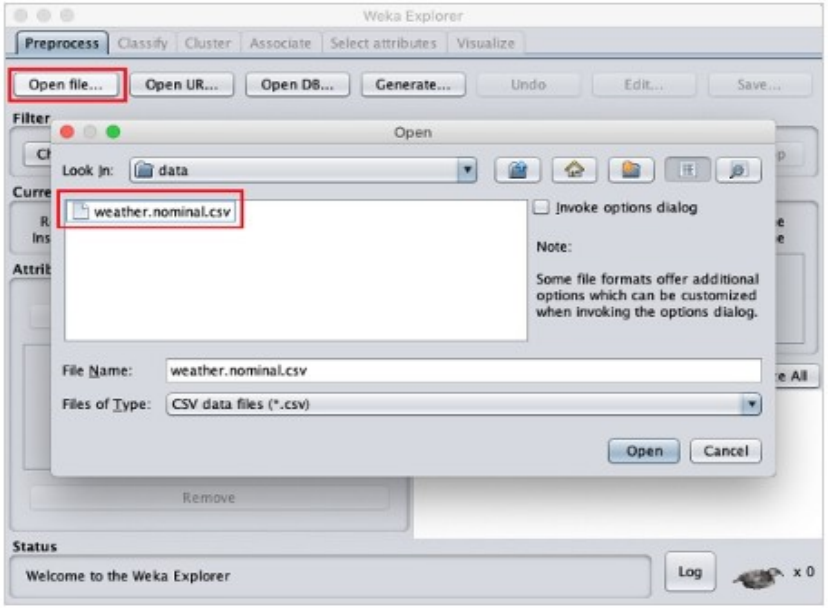
- Local file system
- Web
- Database

Loading Data from Local File System

Just under the Machine Learning tabs that you studied in the previous lesson, you would find the following three buttons –

- Open file ...
- Open URL ...
- Open DB ...

Click on the **Open file** ... button. A directory navigator window opens as shown in the following screen –

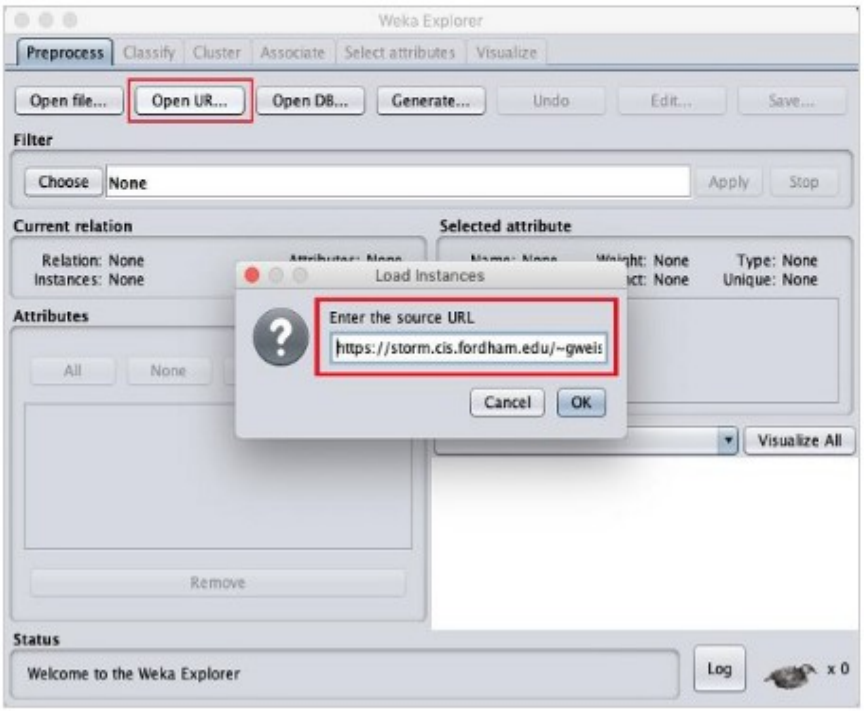


Now, navigate to the folder where your data files are stored. WEKA installation comes up with many sample databases for you to experiment. These are available in the **data** folder of the WEKA installation.

For learning purpose, select any data file from this folder. The contents of the file would be loaded in the WEKA environment. We will very soon learn how to inspect and process this loaded data. Before that, let us look at how to load the data file from the Web.

Loading Data from Web

Once you click on the **Open URL ...** button, you can see a window as follows –



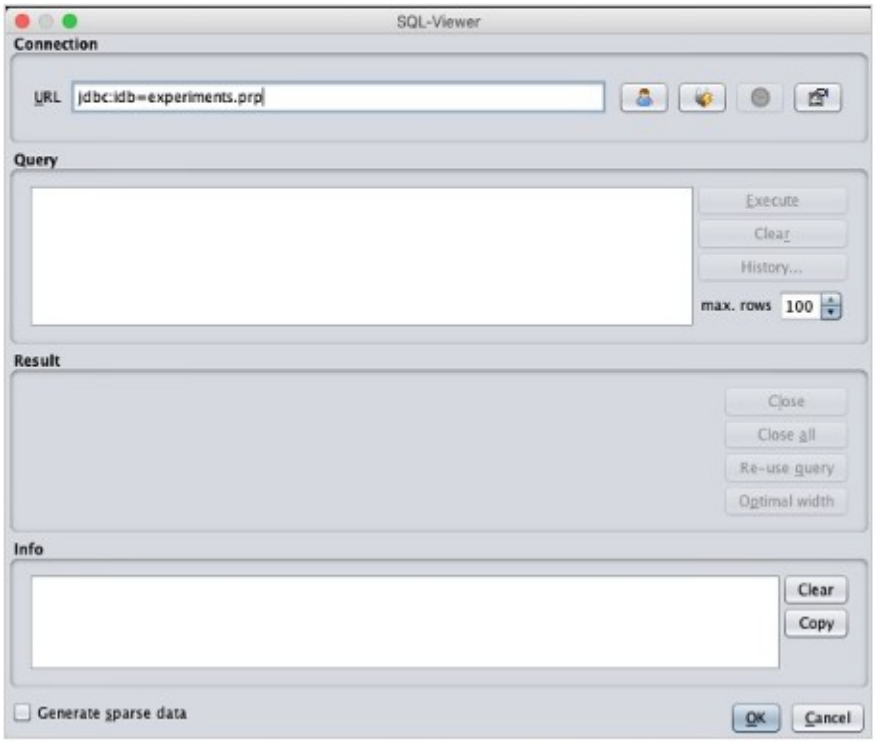
We will open the file from a public URL Type the following URL in the popup box –

https://storm.cis.fordham.edu/~gweiss/data-mining/weka-data/weather.nominal.arff

You may specify any other URL where your data is stored. The **Explorer** will load the data from the remote site into its environment.

Loading Data from DB

Once you click on the **Open DB** ... button, you can see a window as follows –



Set the connection string to your database, set up the query for data selection, process the query and load the selected records in WEKA.

RESULT : Loading of data using weka executed successfully.